

# Versionamento com Git e GitHub

COMP0496 – Programação A

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

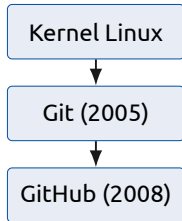
Quantas vezes você já fez isso?

```
trabalho_final.py  
trabalho_final_v2.py  
trabalho_final_AGORA_VAI.py  
trabalho_final_definitivo_USE_ESSE.py
```

**Qual é a versão certa? O que mudou entre elas?**

O Git resolve esse caos: registra cada mudança com data, autor e descrição — para que você nunca perca o histórico do seu trabalho.

- **2005:** Linus Torvalds cria o Git para gerenciar o desenvolvimento do **kernel Linux**
- Motivação: a ferramenta anterior (BitKeeper) revogou a licença gratuita usada pela comunidade Linux
- Em apenas **10 dias** Torvalds escreveu a primeira versão funcional
- Objetivos do projeto: velocidade, design simples, suporte a desenvolvimento não-linear (milhares de branches paralelas) e totalmente distribuído
- **Hoje:** o sistema de controle de versão mais utilizado no mundo



## Git

Sistema de controle de versão distribuído que registra o histórico completo de alterações de um projeto.

**Analogia:** pense nos *checkpoints* de um videogame — você pode voltar a qualquer ponto salvo a qualquer momento.

Três conceitos fundamentais:

- **Repositório** — o projeto junto com todo o seu histórico
- **Commit** — um snapshot salvo com mensagem descritiva
- **Branch** — uma linha de desenvolvimento independente

# Configurando o Git e criando um repositório

```
1  # Configurar identidade (uma vez por máquina)
2  git config --global user.name "Seu Nome"
3  git config --global user.email "seu@email.com"
4
5  # Criar repositório no diretório atual
6  git init meu-projeto
7  cd meu-projeto
```

As três áreas do Git:

**Working Directory** arquivos que você está editando

**Staging Area** arquivos preparados com `git add`

**Repositório** snapshots definitivamente salvos com `git commit`

# Registrando o primeiro commit

```
1  # Criar arquivo hello.py com: print("Ola, mundo!")
2  git status          # ver estado atual
3  git add hello.py    # mover para staging
4  git commit -m "feat: adiciona hello.py"
5  git log --oneline   # ver histórico de commits
```

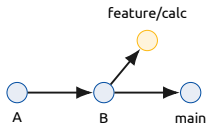
Boas mensagens de commit descrevem *o que* e *por quê* foi alterado, não apenas *como*. Use o prefixo `feat:`, `fix:` ou `docs:` para clareza.

# Trabalhando com branches

## Branch

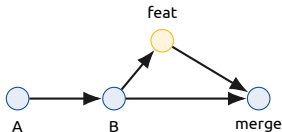
Linha de desenvolvimento independente. Permite trabalhar em uma funcionalidade sem afetar a branch principal (main).

```
1  git branch                      # listar branches
2  git checkout -b feature/calc    # criar e trocar para nova branch
3  # ... editar calculadora.py ...
4  git add calculadora.py
5  git commit -m "feat: adiciona calculadora"
6  git switch main                 # voltar para main
```



Com o trabalho pronto na branch, incorporamos de volta à main:

```
1  # Estando em main:  
2  git merge feature/calc  
3  git branch -d feature/calc    # remover branch mesclada
```



Após o merge, a branch pode ser removida com segurança — seu histórico permanece no repositório via o commit de merge.



# Conflitos de merge

## O que acontece quando dois colaboradores alteram a mesma linha?

O Git insere marcadores no arquivo para indicar as duas versões:

```
<<<<<< HEAD
print("versao da main")
=====
print("versao da feature")
>>>>>> feature/calc
```

## Passos para resolver:

```
1  # 1. Editar o arquivo para manter a versão correta
2  # 2. Marcar como resolvido e commitar
3  git add calculadora.py
4  git commit -m "fix: resolve conflito de merge"
```

## GitHub

Serviço de hospedagem na nuvem para repositórios Git, que facilita compartilhamento, colaboração e revisão de código.

**Distinção importante:** Git é a ferramenta local; GitHub é o servidor remoto na nuvem para compartilhar o repositório.

Após criar o repositório vazio no GitHub (sem README para não conflitar):

```
1 git remote add origin https://github.com/usuario/repo.git
2 git push -u origin main
```

`remote add` associa o repositório remoto; `push -u` envia os commits e configura o *tracking* automático da branch.

# Clonando e colaborando

```
1  # Clonar repositório existente
2  git clone https://github.com/usuario/repo.git
3  cd repo
4
5  # Buscar atualizações do remoto e integrar
6  git pull origin main
```

Fluxo típico de colaboração:

- **clone** — obter cópia completa do repositório
- **branch** — criar branch para a nova funcionalidade
- **editar + commit** — registrar as mudanças localmente
- **push** — enviar a branch para o GitHub
- **Pull Request** — solicitar revisão antes do merge

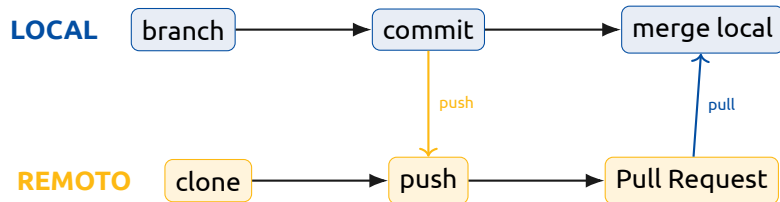
# Pull Requests

## Pull Request

Pedido formal para incorporar uma branch ao projeto, permitindo revisão e discussão antes do merge.

Fluxo completo de um Pull Request:

1. `git checkout -b feature/nova`
2. Fazer commits com as mudanças
3. `git push origin feature/nova`
4. Abrir PR no GitHub (interface web)
5. Colega revisa o código e aprova
6. Merge realizado no GitHub



# Boas práticas

## Commits:

- Pequenos e frequentes — um commit por mudança lógica
- Mensagens no imperativo: "feat: adiciona login"

## Branches e colaboração:

- Nunca commitar diretamente na `main`
- Usar Pull Requests para revisão antes do merge

Exemplo de `.gitignore` para projetos Python:

```
__pycache__/  
*.pyc  
.env  
*.log
```

- **Git** salva o histórico completo — você pode voltar a qualquer versão anterior do projeto
- **Branches** isolam funcionalidades — a `main` fica sempre estável e funcional
- **Conflitos de merge** se resolvem editando o arquivo e commitando normalmente
- **GitHub** é o repositório remoto — `push` envia, `pull` atualiza, PR integra com revisão